

Yaşar University
Spring, 2025 – 2026
SE 2232 - Software System Analysis
Final Project Report: Software Requirements Specifications Document (SRS)

Student Name:	Uday Beyaz & Deniz Avşar
Student No:	23070006061 & 23070006070
Department Name:	Software Engineering
Course Section No:	2

This template is prepared based on the IEEE Recommended Practice for Software Requirements Specifications (IEEE Std 830-1998).

Table of Contents (Do not change the Section Names!)

1	Introduction.....	3
1.1	Purpose	3
1.2	Scope	3
1.3	Definitions, acronyms, and abbreviations	3
1.4	References.....	3
1.5	Overview	4
2	Design and Implementation Constraints.....	4
3	Specific Requirements.....	4
3.1	Functional Requirements	4
3.2	Performance Requirements	5
3.3	Software System Attributes	5
3.4	Use Case Analysis.....	5
3.4.1	Actors	5
3.4.2	Scenarios	6
3.4.3	Use Case Forms.....	6
3.4.4	Relationships among Actors and Use Cases	7
3.4.5	Use Case Diagram	8
4	Behavioral Models.....	8
4.1	Sequence Diagram	9
5	Structural Models.....	11
5.1	Class Diagram	11
6	Process Modeling	12
6.1	Data Flow Diagram (DFD)	12
7	Graphical User Interface(s) (GUIs)	13
8	Conclusion and Future Work	25

1 Introduction

No explanation is needed here. Only complete the subsections.

1.1 Purpose

The purpose of this SRS is to define the full requirements of the "MovieCritics" application. It covers how the system manages a movie database, handles different user roles (Parent/Child), and ensures secure data persistence using SQLite.

1.2 Scope

MovieCritics is a Java Swing desktop application for families to manage their movie libraries.

- **What it does:** It lets Parents (Type-1) manage all movies and users, while Children (Type-2) can only see allowed movies, rate them, and add them to a watchlist.
- **Goal:** To provide a safe, shared cinematic experience for the whole family while keeping restricted content away from kids.

1.3 Definitions, acronyms, and abbreviations

- **SRS:** Software Requirements Specification
- **DAO:** Data Access Object (The pattern we used in `MovieDAO` to handle SQL)
- **Type-1 User:** Parent/Admin with full access
- **Type-2 User:** Child/Member with restricted access
- **JDBC:** Java Database Connectivity (The bridge between our Java code and SQLite)

1.4 References

- IEEE Std 830-1998 for SRS documents.
- Official SQLite Documentation.
- Java Swing API Documentation.

1.5 Overview

This document consists of 8 sections. It begins with design constraints, then moves on to functional requirements, use case analysis, UML diagrams (Sequence, Class, Data Flow Diagram), and finally shows GUI screenshots.

2 Design and Implementation Constraints

- **Language:** Java SE 17+.
- **IDE:** IntelliJ IDEA.
- **Database:** SQLite (using `sqlite-jdbc` driver).
- **Modeling:** Visual Paradigm for all diagrams.
- **Connectivity:** All database calls must go through `DatabaseManager.getConnection()` to avoid "pointer closed" errors.

3 Specific Requirements

No explanation is needed here. Only complete the subsections.

3.1 Functional Requirements

- **FR1:** The system shall provide a login screen to authenticate users.
- **FR2:** The system shall hide movies marked as `ParentalRestriction = 1` from Type-2 users.
- **FR3:** The system shall allow Type-1 users to Add, Edit, and Delete movies using the `MovieDAO`.
- **FR4:** The system shall allow Type-2 users to mark movies as "Watched" and give ratings from 1 to 10.
- **FR5:** The system shall maintain a personal watchlist for each Child user.

3.2 Performance Requirements

- **PR1:** The system shall load the filtered movie list in less than 2 seconds.
- **PR2:** The database shall handle concurrent read/write operations without data corruption.
- **PR3:** GUI transitions between Login and Main frames should be seamless.

3.3 Software System Attributes

- **Reliability:** The system shall commit all database transactions atomically, ensuring that no partial writes occur; in the event of an unexpected application shutdown, no data corruption shall occur in the SQLite database file.
- **Availability:** The application shall be available for use at any time on any desktop computer where the Java Runtime Environment and the SQLite JDBC driver are properly installed, without requiring an internet connection.
- **Security:** The system shall restrict access to parent-level functions (add/remove movies, manage users, view analytics) exclusively to Type-1 users; Type-2 users shall not be able to access any admin panel through the GUI.
- **Maintainability:** The application code shall be organized into separate Java classes by responsibility (DatabaseManager, MovieDAO, model classes), so that any single class can be modified without affecting unrelated components.
- **Usability:** The application shall provide clear error messages and confirmation dialogs for all destructive actions, and all forms shall validate required fields before attempting a database operation.

3.4 Use Case Analysis

No explanation is needed here. Only complete the subsections.

3.4.1 Actors

- **Parent (Type-1):** Admin who manages the library and users.
- **Child (Type-2):** User who browses, rates, and watches movies.
- **Database:** The SQLite engine that stores all info.

3.4.2 Scenarios

- **Scenario 1 (Parent adds a new restricted movie):** Maria (Type-1) logs in with her parent account. She opens the "Add Movie" dialog and enters the details for "The Dark Knight". She checks the "Parental Restriction" box because the film is not suitable for young children. After saving, the movie is added to the database but is invisible to her children when they log in.
- **Scenario 2 (Child rates a movie):** Tom (Type-2) logs in with his child account and browses the allowed movies. He selects "Inception" and uses the "Rate Movie" function to give it a score of 9 out of 10. He also adds a comment saying "Great visuals!". His sister Sarah can later see this rating in the "Family Ratings" screen.

3.4.3 Use Case Forms

Complete Use Case 1: Login

- **Use case name:** Login
- **Participating actors:** Parent (Type-1), Child (Type-2)
- **Description:** Authenticates the user and opens the specific interface based on their role.
- **Trigger:** User launches the application.
- **Preconditions:** User has a registered account in the Users table.
- **Normal course:** 1. User enters username and password. 2. System verifies credentials via DatabaseManager. 3. System opens Type1MainFrame or Type2MainFrame.
- **Post conditions:** User is successfully logged in to the correct dashboard.
- **Exceptions:** If credentials are wrong, an "Invalid username or password" dialog appears.

Complete Use Case 2: Add Movie

- **Use case name:** Add Movie
- **Participating actors:** Parent (Type-1)
- **Description:** Allows a parent to add a new movie record to the shared library.
- **Trigger:** User selects "Add Movie" from the menu.
- **Preconditions:** Parent is authenticated.
- **Normal course:** 1. AddEditMovieDialog opens. 2. User fills in movie details. 3. User clicks Save. 4. MovieDAO executes the SQL insert.
- **Post conditions:** A new record is created in the Movie table.
- **Exceptions:** If required fields are empty, a validation error is shown.

Complete Use Case 3: View Family Ratings

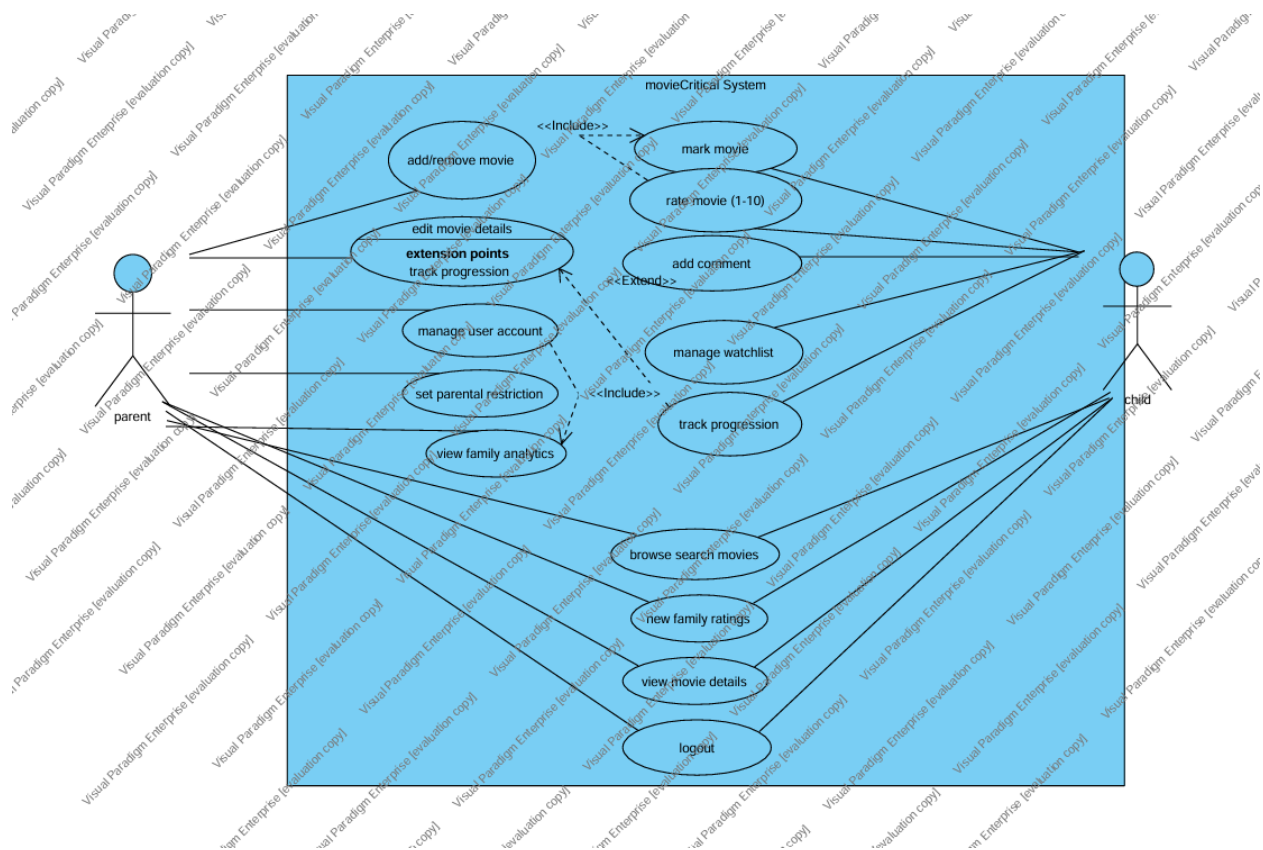
- **Use case name:** View Family Ratings
- **Participating actors:** Parent (Type-1), Child (Type-2)

- **Description:** Users can view scores and comments left by other family members for a specific movie.
- **Trigger:** User clicks "View Family Ratings" button.
- **Preconditions:** The selected movie must exist in the database.
- **Normal course:** 1. FamilyRatingsDialog opens. 2. User selects a movie. 3. System fetches and displays ratings from the UserRating table.
- **Post conditions:** User sees the list of family reviews.

3.4.4 Relationships among Actors and Use Cases

- The **Parent (Type-1)** is the primary administrator and is associated with all library and user management use cases.
- The **Child (Type-2)** is a restricted user associated with browsing, rating, and watchlist activities for non-restricted movies.
- The **SQLite Database** acts as a secondary actor that provides persistent storage for all use cases.

3.4.5 Use Case Diagram

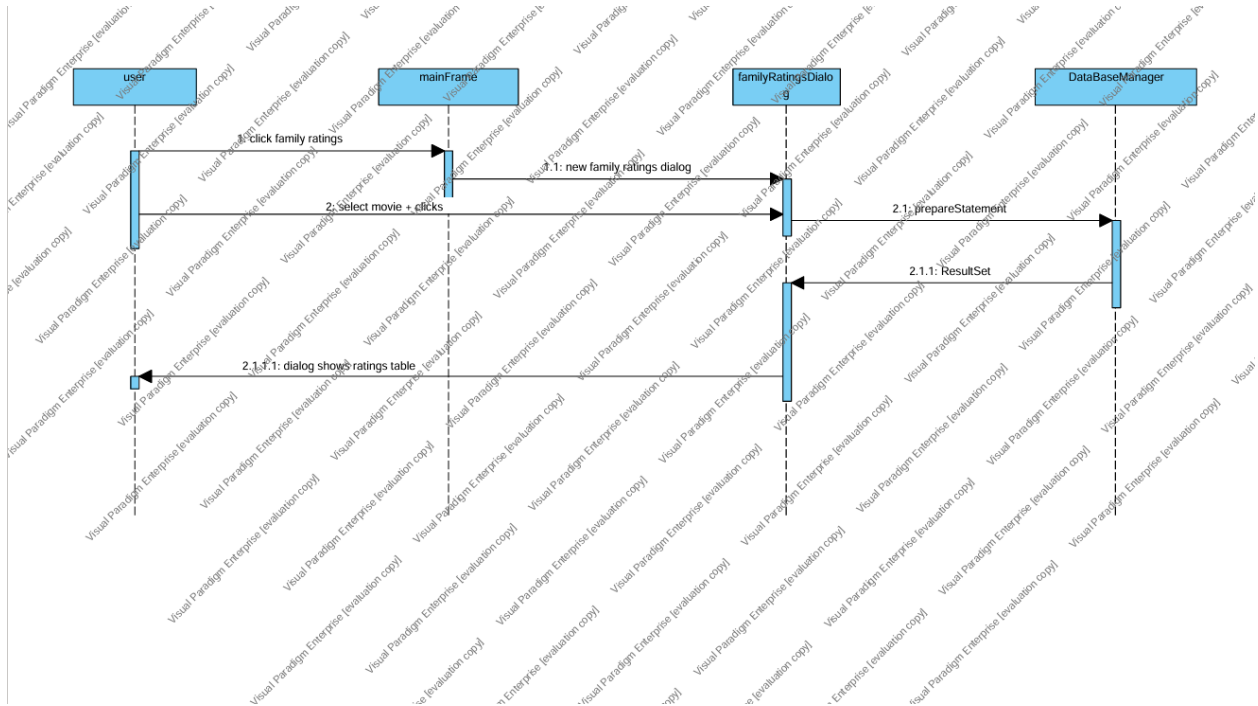


The use case diagram above illustrates the interactions between the two actors of the MovieCritics system and the system's functionalities. The Parent (Type-1) actor is associated with all administrative use cases including Add/Remove Movie, Edit Movie, Manage User Accounts, Set Parental Restriction, Moderate Content, and View Family Analytics. The Child (Type-2) actor is associated with member-level use cases including Mark Movie as Watched, Rate Movie, Add Comment, Manage Watchlist, and Track Progress. Both actors share the Browse/Search Movies, View Movie Details, View Family Ratings, and Logout use cases.

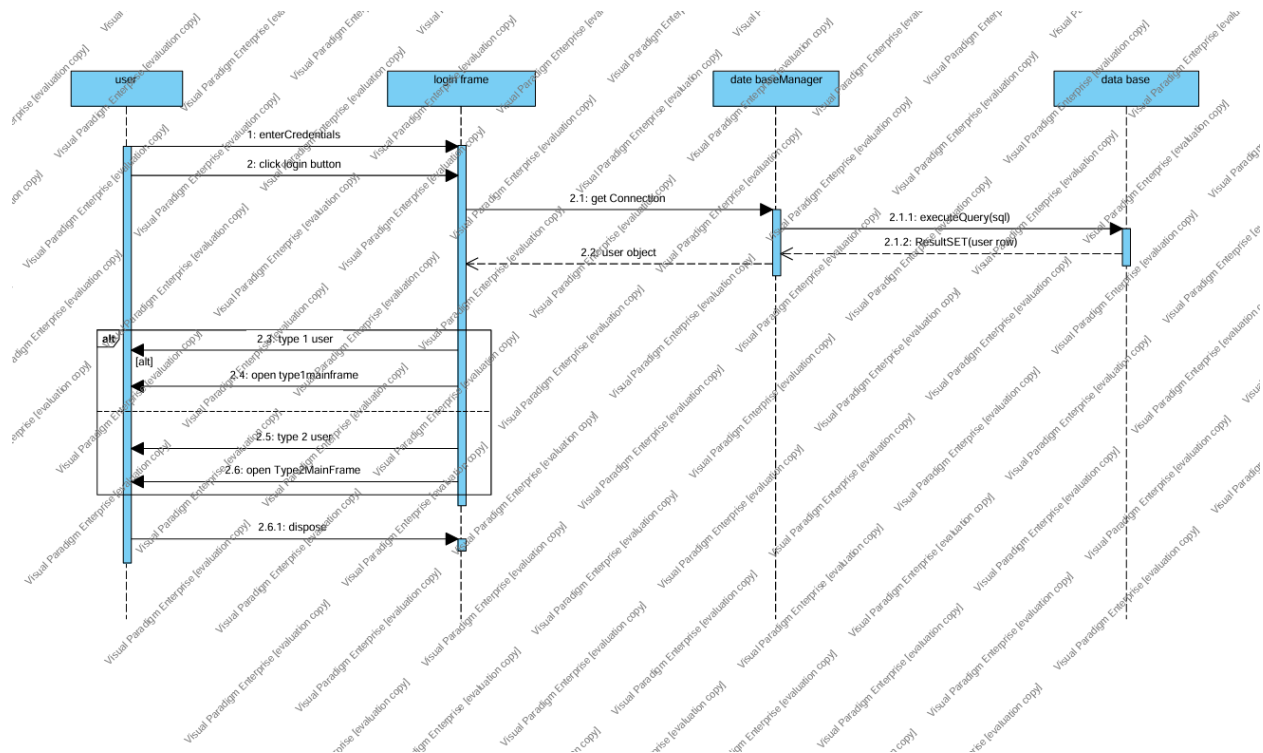
4 Behavioral Models

No explanation is needed here. Only complete the subsection.

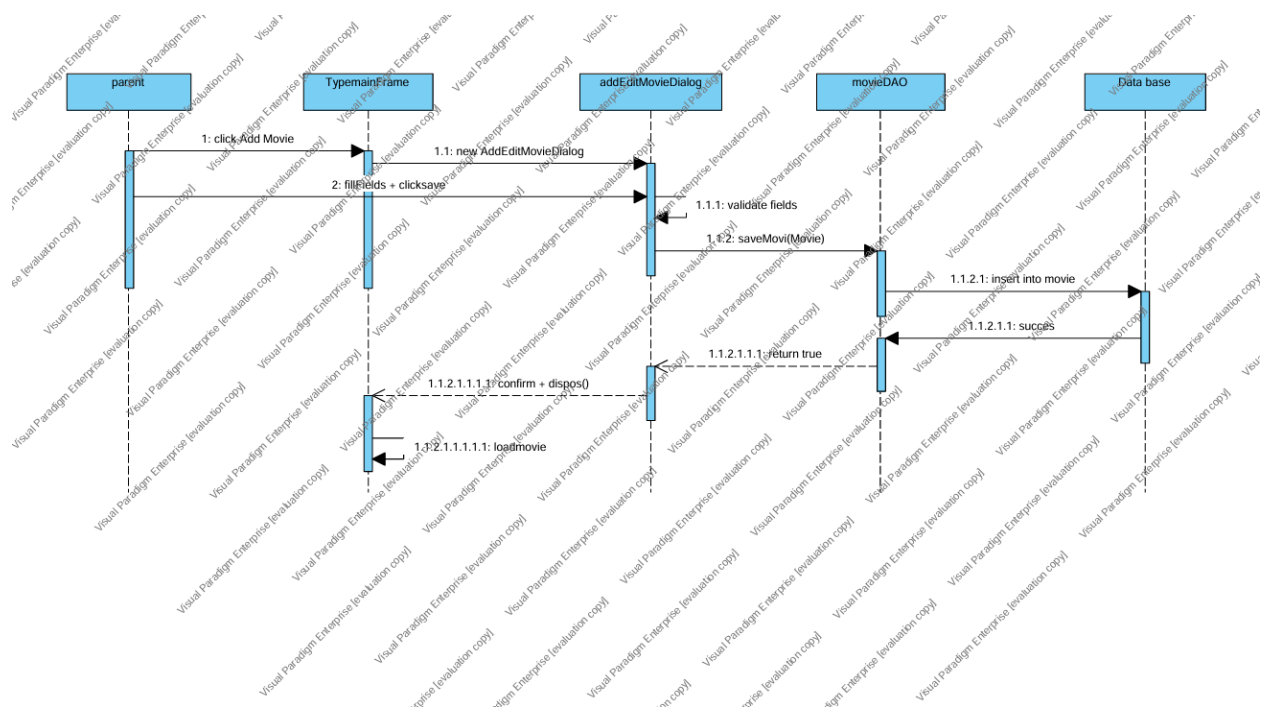
4.1 Sequence Diagram



This sequence diagram illustrates the View Family Ratings use case. Any authenticated user selects View Family Ratings from the menu, which opens the FamilyRatingsDialog. The user selects a movie from the dropdown and clicks Show Ratings. The dialog calls DatabaseManager to execute a JOIN query across the UserRating, Users, and Movie tables. The resulting data is displayed in a table showing each family member's username, user type, score, and comment for the selected movie.



This sequence diagram illustrates the Login use case. When the user enters their credentials and clicks the Login button, the LoginFrame calls DatabaseManager.getConnection() to establish a database connection and executes a prepared SQL statement to verify the credentials against the Users table. If the credentials match a Type-1 user, Type1MainFrame is opened; if they match a Type-2 user, Type2MainFrame is opened. In both cases, the LoginFrame is disposed. If the credentials are invalid, an error message is displayed and the LoginFrame remains open.

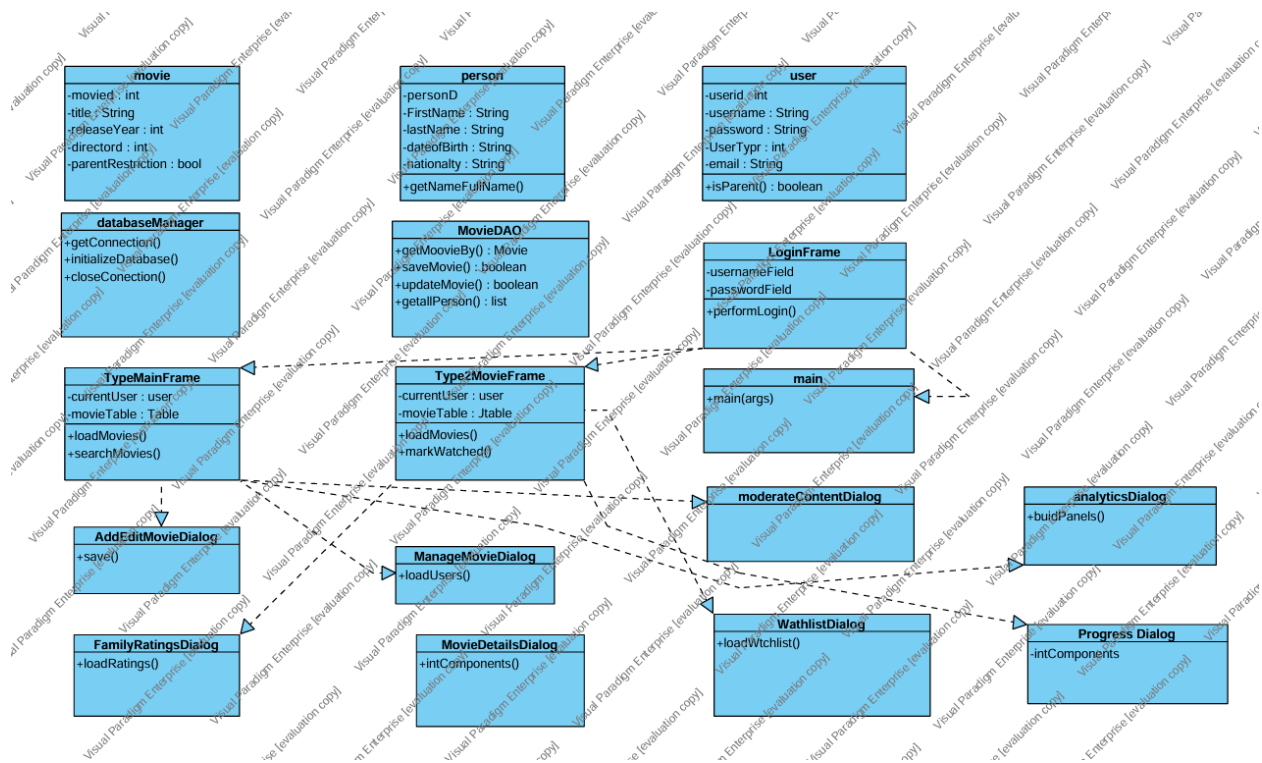


This sequence diagram illustrates the Add Movie use case. The Parent user selects Add Movie from the menu in Type1MainFrame, which opens the AddEditMovieDialog. The user fills in all required fields and clicks Save. The dialog validates the inputs and calls MovieDAO.saveMovie(), which uses DatabaseManager.getConnection() to execute an INSERT SQL statement into the Movie table. Upon success, the dialog closes and Type1MainFrame calls loadMovies() to refresh the movie list.

5 Structural Models

No explanation is needed here. Only complete the subsection.

5.1 Class Diagram

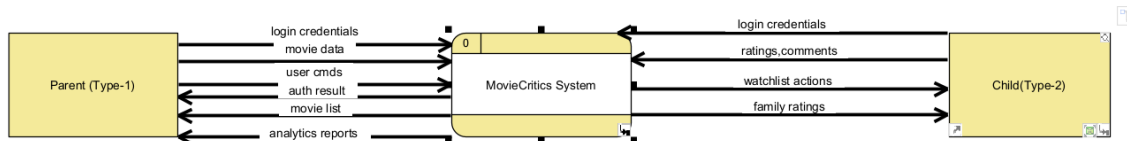


The class diagram above shows the complete structure of the MovieCritics application organized into four layers. The Model layer contains the Movie, Person, and User classes which hold all data attributes and provide getters and setters. The Data Access layer contains DatabaseManager, which manages the SQLite JDBC connection and initializes the database, and MovieDAO, which provides static CRUD methods for movie operations. The View/Frame layer contains LoginFrame, Type1MainFrame, Type2MainFrame, and Main, which form the entry point and main windows of the application. The Dialog layer contains eight JDialog subclasses: AddEditMovieDialog, ManageUsersDialog, ModerateContentDialog, AnalyticsDialog, FamilyRatingsDialog, MovieDetailsDialog, WatchlistDialog, and ProgressDialog, each responsible for a specific user interaction. Dependency relationships are shown with dashed arrows indicating which classes create or use other classes.

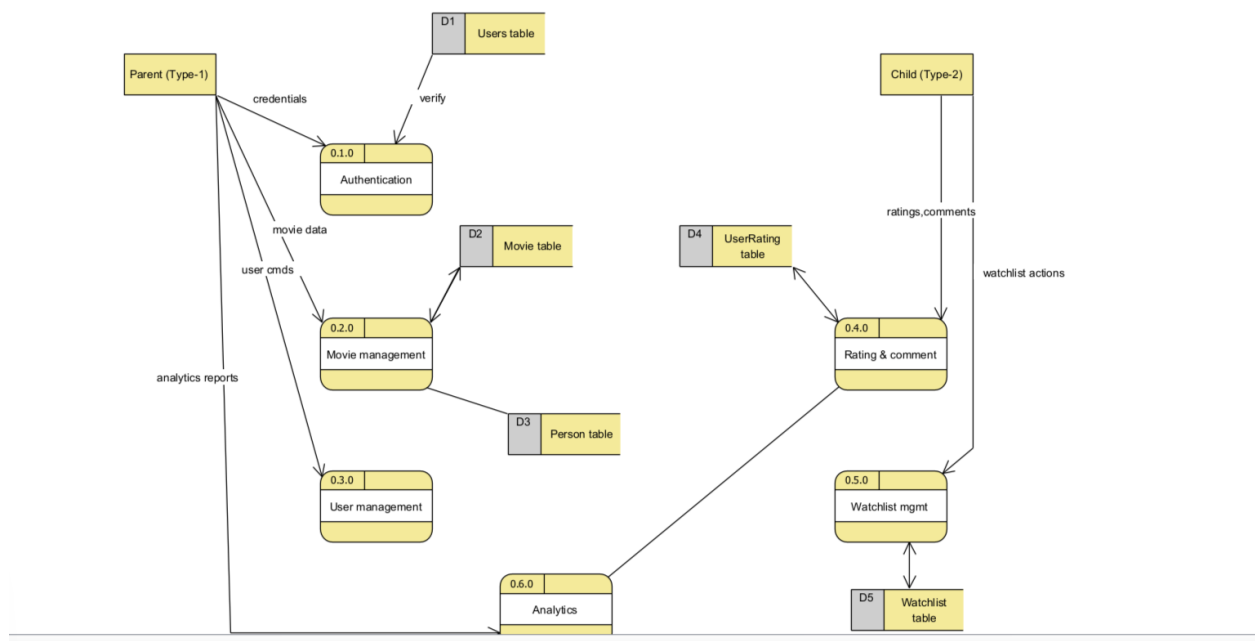
6 Process Modeling

No explanation is needed here. Only complete the subsection.

6.1 Data Flow Diagram (DFD)



The context diagram above provides the highest-level view of the MovieCritics system. It contains a single process (Process 0) representing the entire system, and two external entities: Parent (Type-1) and Child (Type-2). The Parent entity sends login credentials, movie data, and user management commands to the system, and receives authentication results, the full movie list, and analytics reports in return. The Child entity sends login credentials, ratings, and comments to the system, and receives a filtered movie list (excluding restricted movies), family ratings, watchlist data, and progress information in return. No data stores are shown in the context diagram as per DFD conventions.



The Level 0 DFD decomposes the MovieCritics system into six major processes. Process 1 (Authenticate User) verifies credentials against the D1 Users data store and redirects the user to the correct interface. Process 2 (Manage Movies) handles all movie CRUD operations, reading and writing to the D2 Movie data store and reading person data from D3 Person. Process 3 (Manage Users) allows the Parent to create, edit, and delete user accounts, interacting with D1 Users. Process 4 (Rate and Comment) allows the Child to submit ratings and comments, storing them in D4 UserRating. Process 5 (Manage Watchlist) allows the Child to add and remove movies from their personal watchlist, stored in D5 Watchlist. Process 6 (Generate Analytics) collects data from Processes 2 and 4 and delivers analytics reports back to the Parent user.

7 Graphical User Interface(s) (GUIs)

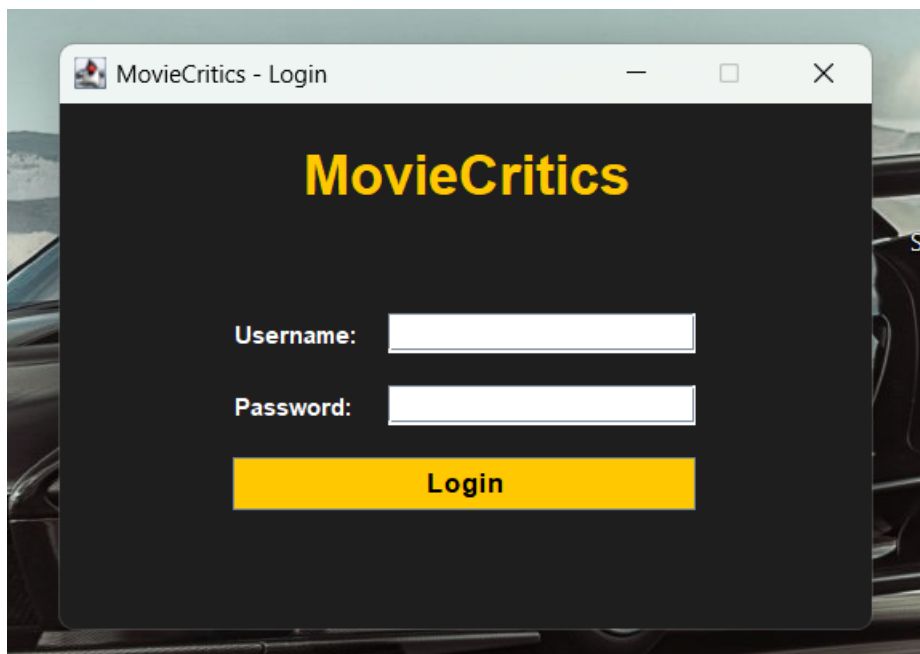


Figure 1: Login Interface

Purpose: This is the initial entry point where all users are authenticated. It features a secure login system that handles credentials to redirect users based on their specific roles (Parent or Child).

ID	Title	Year	Genre	Director	Watched	Rating	Parental
1	Inception	2010	Sci-Fi	Christopher Nolan	Yes	9	No
2	Interstellar	2014	Sci-Fi	Christopher Nolan	No	9	No
5	Saving Private Ryan	1998	War	Steven Spielberg	Yes	9	Yes
3	Schindler's List	1993	Drama	Steven Spielberg	Yes	10	Yes
4	The Martian	2015	Sci-Fi	Ridley Scott	No	8	No

Figure 2: Parent/Admin Main Dashboard (Type-1)

Purpose: The central management hub for Parent users.

Functionality: It displays the full list of movies and provides access to administrative tools like adding, editing, or removing movies and managing users.

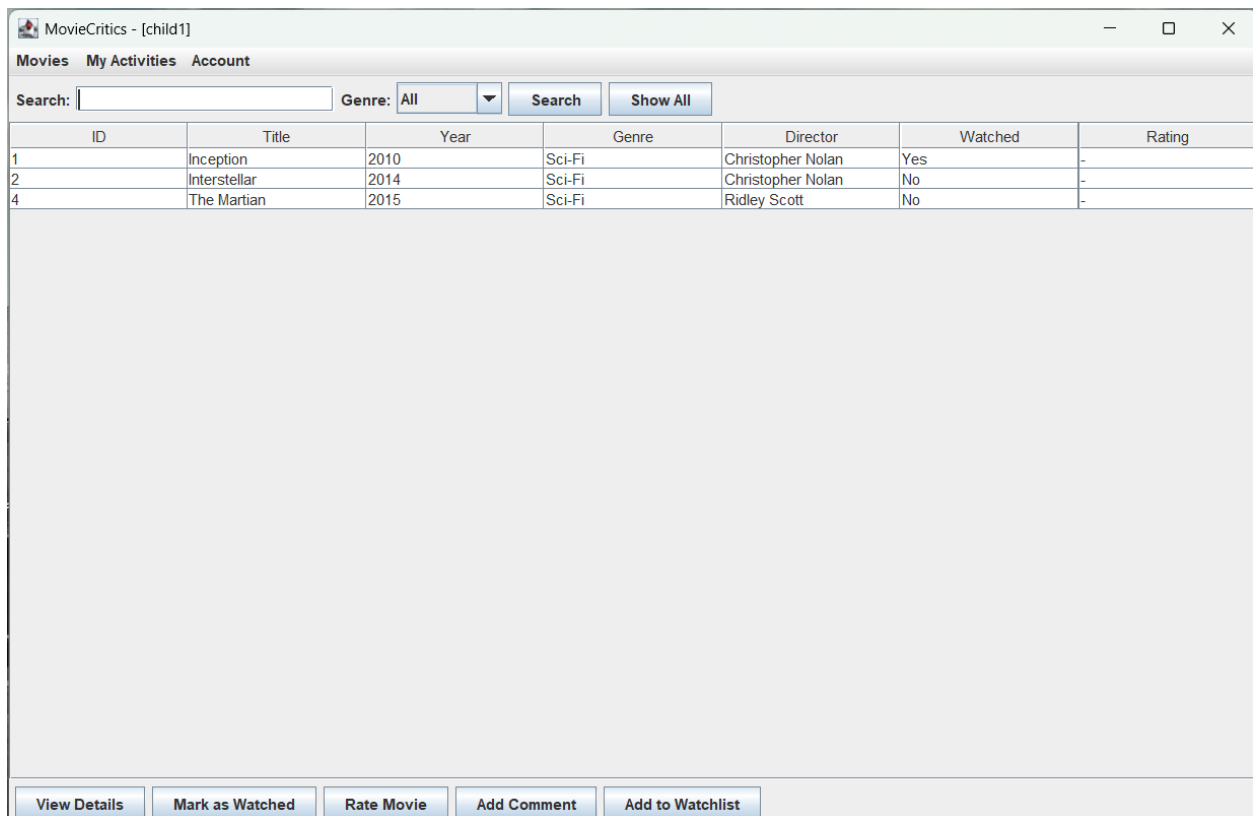


Figure 3: Child/Member Main Dashboard (Type-2)

Purpose: The restricted viewing area for younger family members.

Functionality: It displays only movies where ParentalRestriction is set to 0 (False). It includes buttons for rating, commenting, and managing a personal watchlist.

Add Movie

Title:

Release Year:

Language:

Country:

Genre:

Director:

Lead Actor:

Support Actor:

Rating (1-10):

Poster URL:

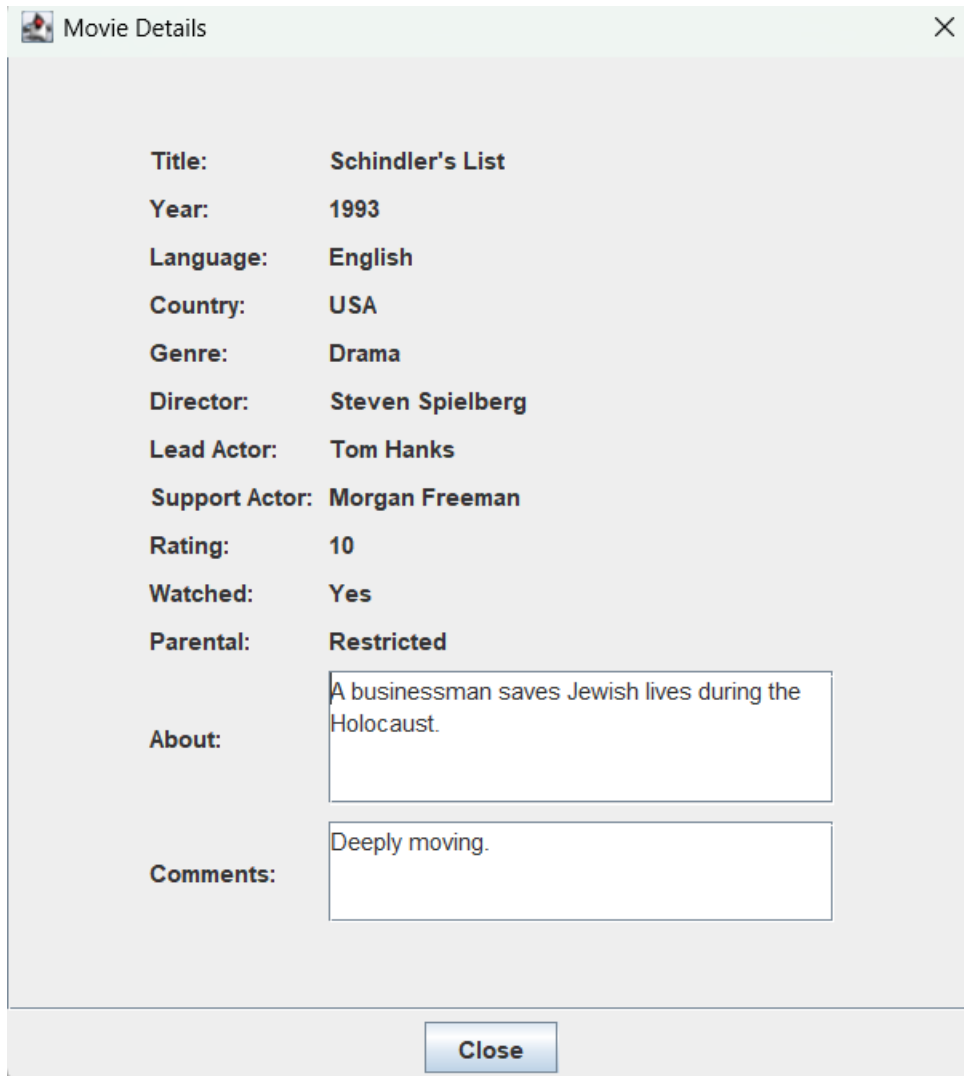
About:

Comments:

☒ Watched ☐ Parental Restriction

Figure 4: Add New Movie Dialog Test

Description: This screenshot demonstrates the movie addition functionality. The parent user can input comprehensive metadata, including parental restriction flags, which are then saved to the SQLite database.



A screenshot of a 'Movie Details' dialog box. The dialog has a title bar with a small icon and the text 'Movie Details', and a close button (X) in the top right corner. The main content area is light gray and contains a list of movie details on the left and their corresponding values on the right. The details include Title, Year, Language, Country, Genre, Director, Lead Actor, Support Actor, Rating, Watched status, and Parental rating. Below these are two text boxes: 'About' and 'Comments'. The 'About' box contains the text 'A businessman saves Jewish lives during the Holocaust.' and the 'Comments' box contains 'Deeply moving.' At the bottom center of the dialog is a 'Close' button.

Title:	Schindler's List
Year:	1993
Language:	English
Country:	USA
Genre:	Drama
Director:	Steven Spielberg
Lead Actor:	Tom Hanks
Support Actor:	Morgan Freeman
Rating:	10
Watched:	Yes
Parental:	Restricted
About:	A businessman saves Jewish lives during the Holocaust.
Comments:	Deeply moving.

Close

Figure 5: Family Ratings and Reviews Test

Description: This dialog displays the collaborative nature of the system. It shows a list of ratings and comments retrieved from the UserRating table, allowing family members to see each other's opinions.

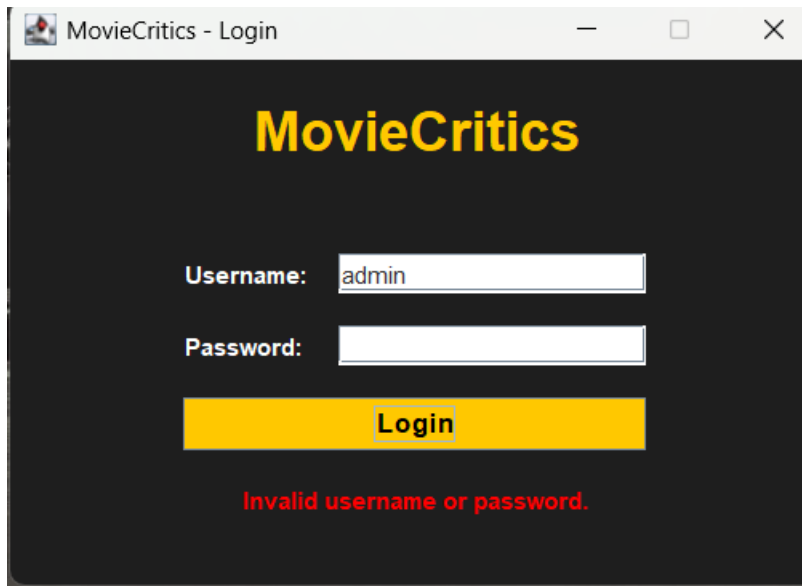


Figure 6: Error handling for unauthorized access attempts.

- **Description:** This screenshot illustrates the application's security layer and exception handling. When a user provides incorrect credentials, the system triggers a JOptionPane error dialog instead of crashing or allowing access. This validates the robust authentication logic implemented in the backend to protect sensitive administrative data.

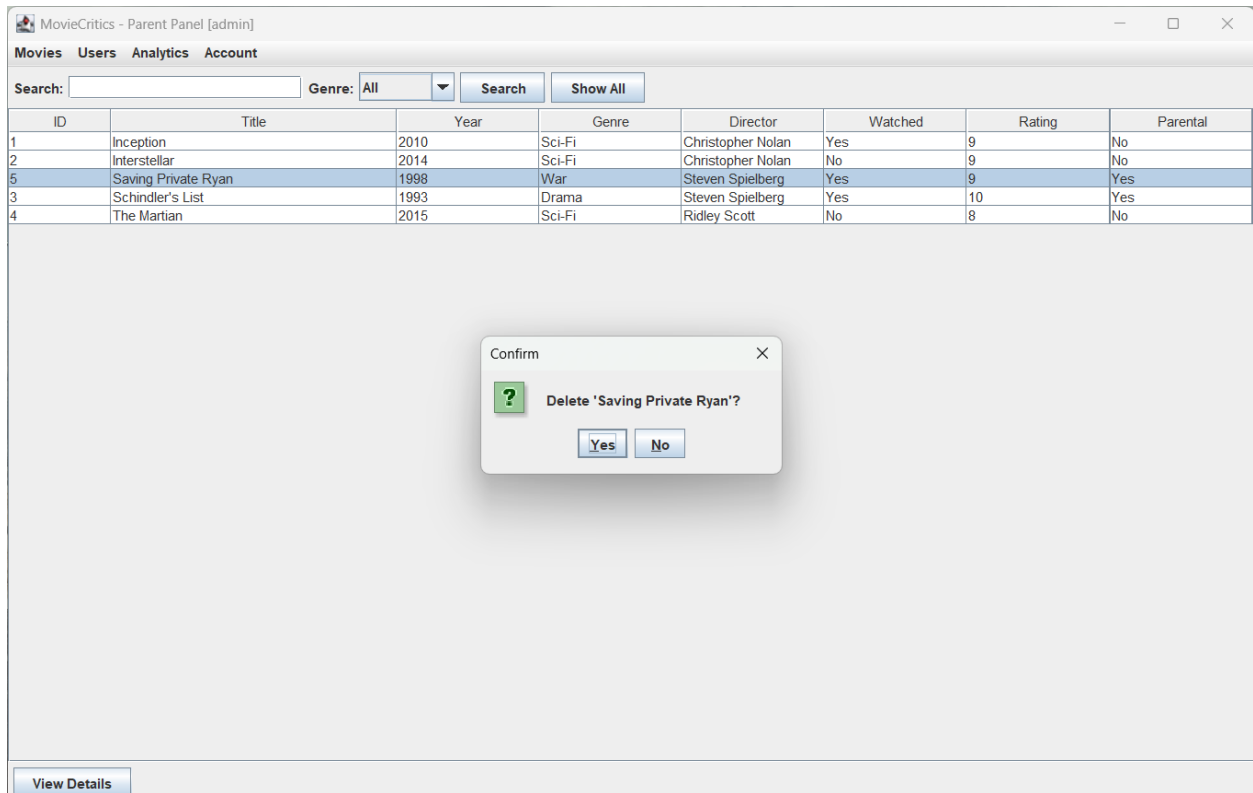
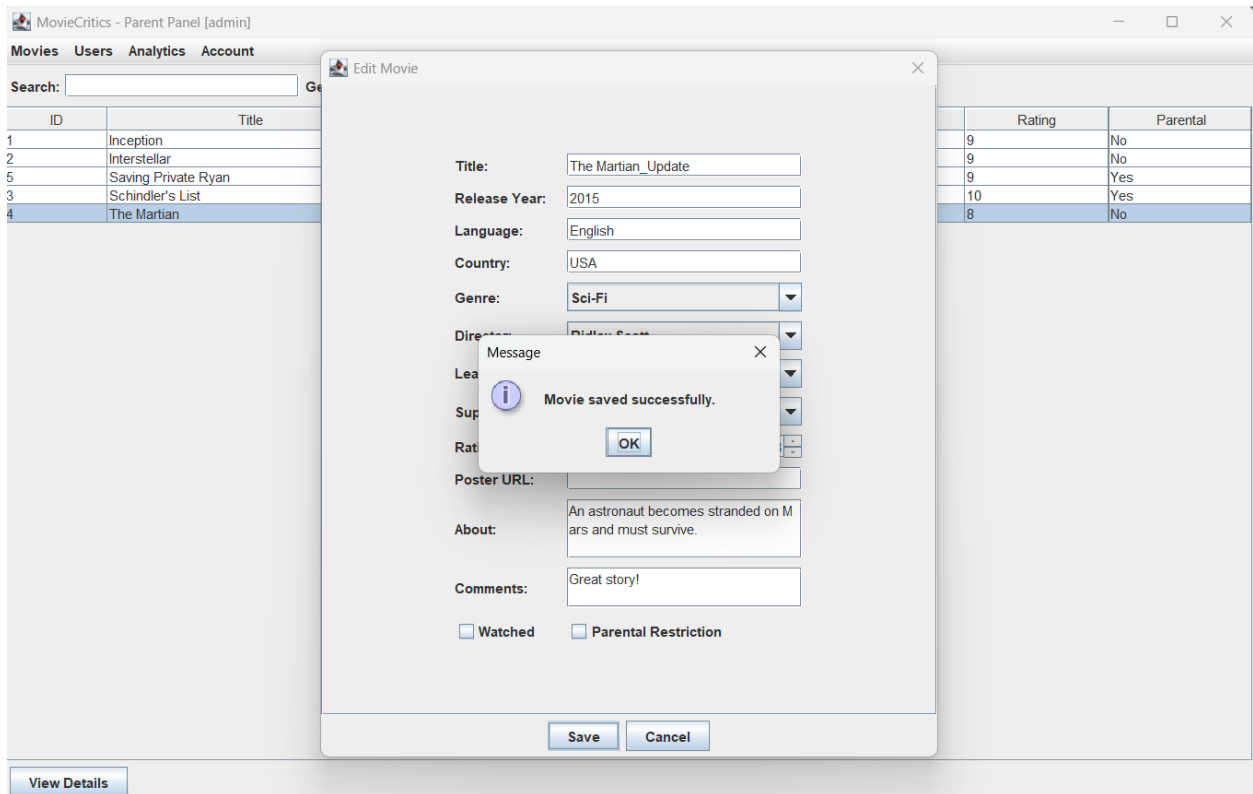
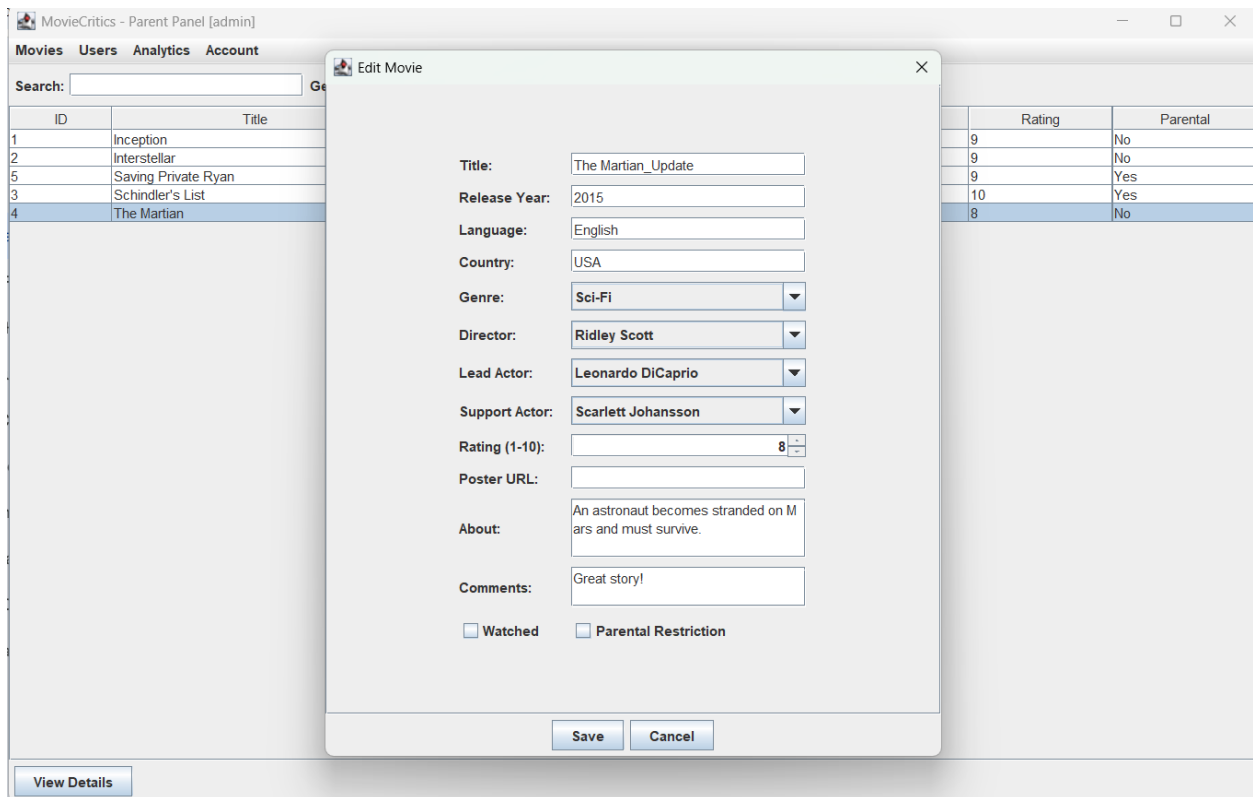


Figure 7: Confirmation dialog for record deletion.

- **Description:** To ensure data integrity, the system implements a defensive programming approach. Before executing a DELETE query via the MovieDAO, the UI prompts the Parent with a confirmation modal. This prevents the accidental loss of movie records and ensures that destructive database operations are intentional.



MovieCritics - Parent Panel [admin]

Movies Users Analytics Account

Search: Genre: All

ID	Title	Year	Genre	Director	Watched	Rating	Parental
1	Inception	2010	Sci-Fi	Christopher Nolan	Yes	9	No
2	Interstellar	2014	Sci-Fi	Christopher Nolan	No	9	No
5	Saving Private Ryan	1998	War	Steven Spielberg	Yes	9	Yes
3	Schindler's List	1993	Drama	Steven Spielberg	Yes	10	Yes
4	The Martian_Update	2015	Sci-Fi	Ridley Scott	No	8	No

Figure 8: Demonstration of the Edit/Update Workflow. Description: These screenshots provide a step-by-step validation of the system's UPDATE capability. The first image shows the Parent user modifying a movie's metadata (e.g., Title or Director) within the AddEditMovieDialog. The second image confirms the success of the database transaction; after clicking 'Save', the system executes a parameterized SQL update through the MovieDAO, and the main table is immediately refreshed to reflect the new persistent data stored in the SQLite database.

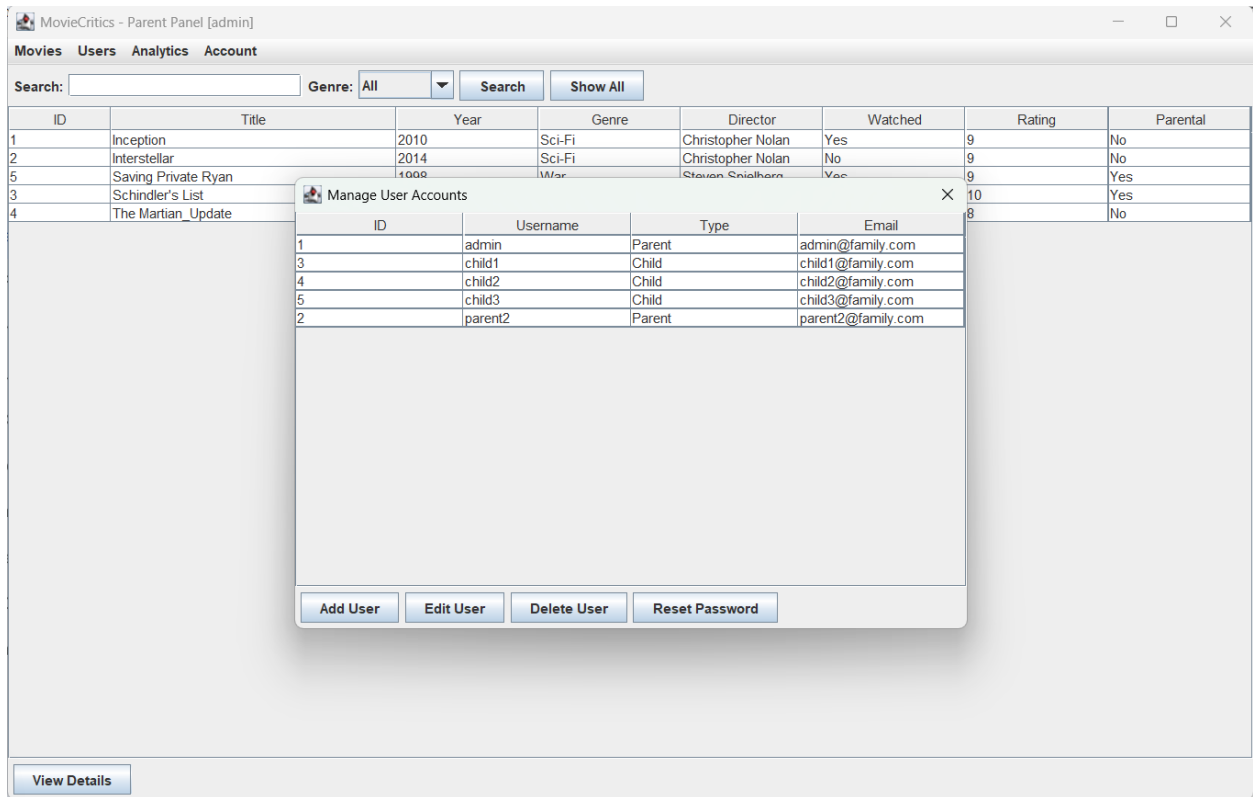


Figure 9: User Management panel for role-based control.

- Description:** This is the core of our "Parental Control" feature. The panel displays all registered accounts fetched from the Users table. It allows the Parent to assign roles (Type-1 or Type-2), which globally dictates what content and functions are accessible to that specific family member.

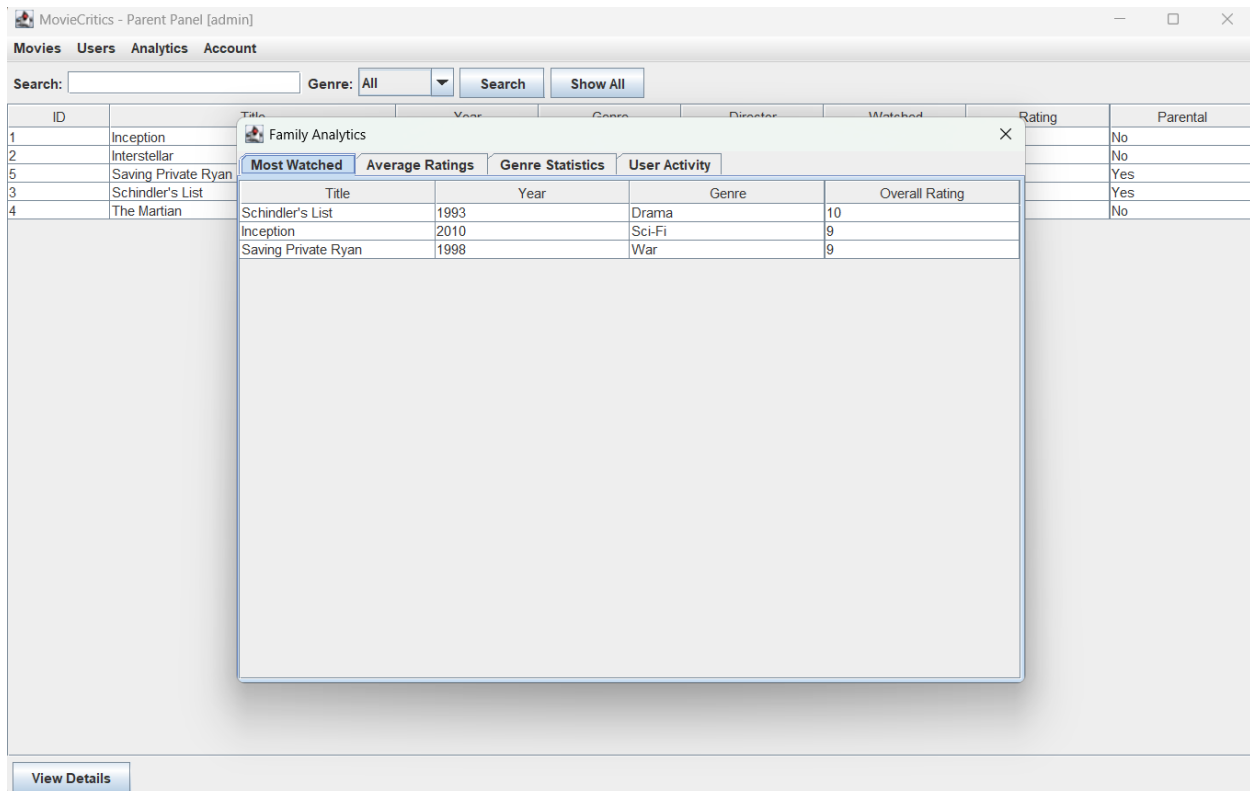


Figure 10: Statistical overview of family analytics.

- Description:** The Analytics dashboard provides a high-level summary of system usage. It utilizes SQL aggregate functions (like AVG and COUNT) to present data such as the most popular genres or average family ratings, helping parents understand the viewing preferences within the household.

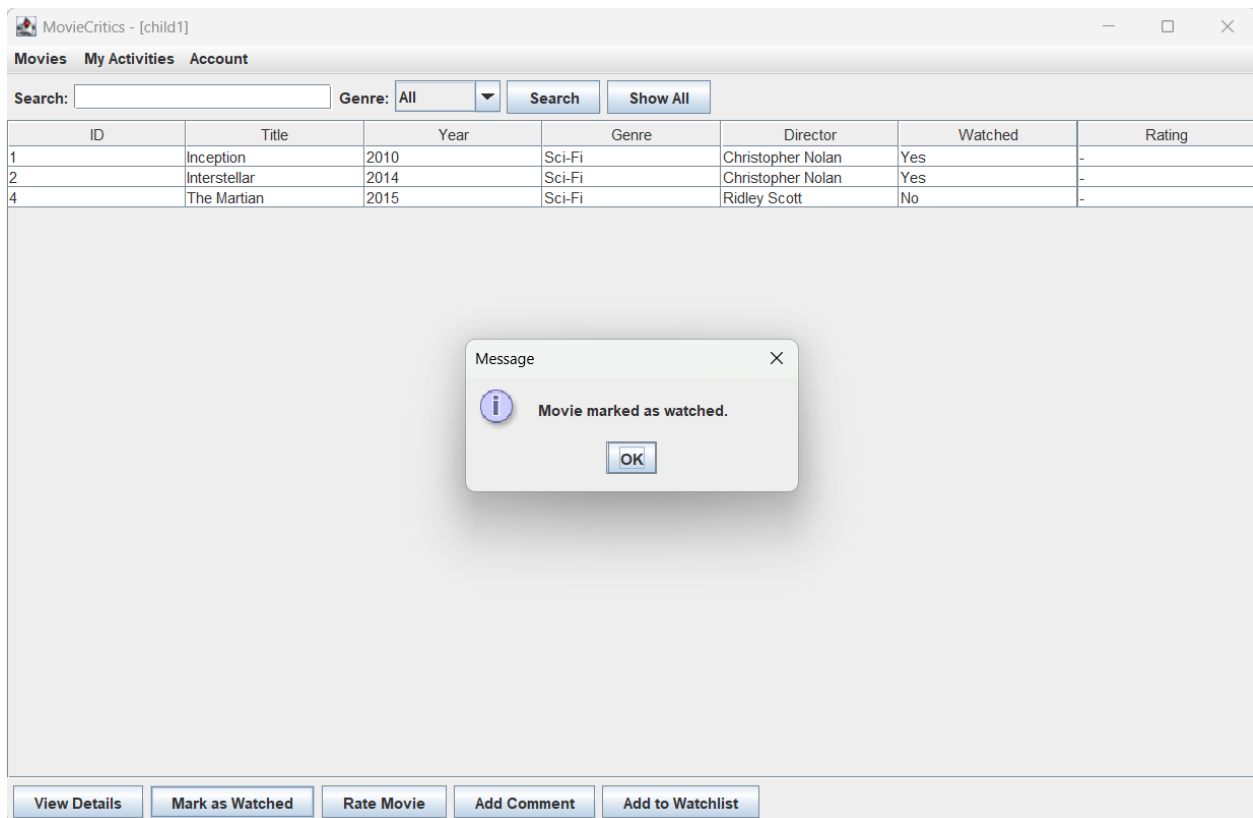


Figure 11: Feature for tracking personal viewing history.

- **Description:** This screenshot shows the interactive "Watched" status toggle. When a Child marks a movie as watched, the system updates a boolean flag in the database. This allows for personalized content tracking, enabling children to easily distinguish between new and previously seen movies.

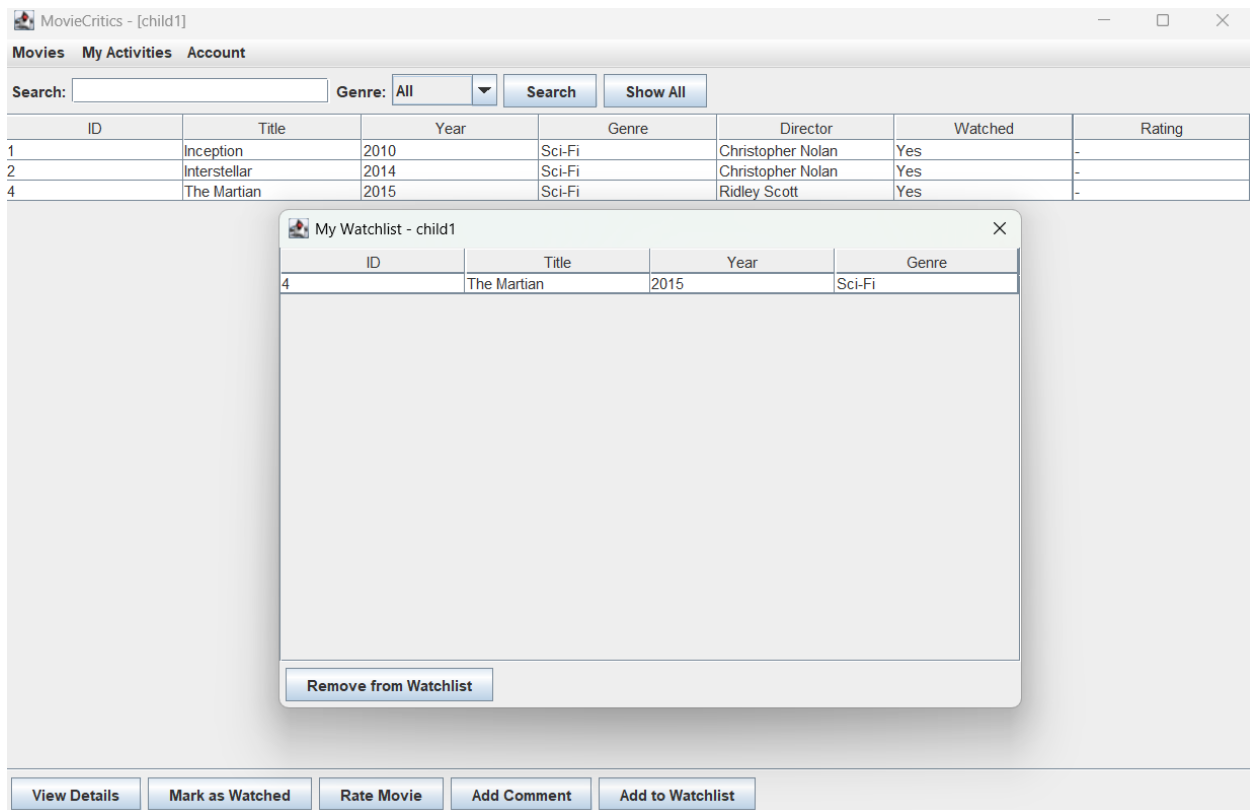


Figure 12: User-specific watchlist functionality.

- Description:** The Watchlist feature serves as a personalized queue for Child users. This screen proves that the application can handle many-to-many relationships in the database, where multiple users can save various movies to their own private lists for future sessions.

8 Conclusion and Future Work

- Conclusion:** The MovieCritics application effectively demonstrates a role-based access control system for family movie management. By utilizing Java Swing for the GUI and SQLite for persistent data storage, we created a functional desktop environment where parental control is seamlessly integrated. We successfully addressed database connection challenges (such as closed statement pointers) by optimizing the DatabaseManager class.
- Future Work:** Future enhancements could include integrating an external API (like OMDB) to automatically fetch movie data and posters. Additionally, implementing password hashing

would improve security, and adding a recommendation engine based on user ratings would enhance the user experience.